Safe Proximal Policy Optimization for Sparse-Reward Classic Control: A PPO-Lagrangian Study on `MountainCar`

Hadis Surya
hadisssurya@gmail.com

Abstract

We study constrained reinforcement learning in the sparse-reward classic control task `MountainCar-v0`. We progressively build three agents: (i) a Deep Q-Network (DQN) baseline, (ii) a Proximal Policy Optimization (PPO) baseline, and (iii) *Safe PPO*, a dual-critic PPO-Lagrangian agent that respects a soft proportional speed constraint expressed in a Constrained Markov Decision Process (CMDP). Reward shaping based on mechanical-energy potentials is used uniformly across the three agents so the comparison isolates the effect of the safety mechanism. Safe PPO augments the standard actor–critic with a second critic that estimates the cost-value function, and adapts the Lagrangian multiplier λ via dual ascent against a cost budget d. On `MountainCar-v0` with a maximum-speed constraint $|\dot{x}| \leq 0.04$ and budget d=15, Safe PPO reaches a return comparable to unconstrained PPO (≈ 48 vs ≈ 47 shaped return) while satisfying the soft cost budget in steady state. The trajectory of λ tracks the violation magnitude, demonstrating that the multiplier acts as an interpretable, automatic safety regulator. We release the full code, configurations, and analysis scripts.

1 Introduction

Reinforcement learning (RL) has produced impressive empirical results across games and control [5,10], but standard RL implicitly assumes that the agent is free to maximize reward irrespective of side effects. Many practical applications — robotics, autonomous driving, healthcare — require the agent to satisfy additional safety or operational constraints. Constrained Markov Decision Processes (CMDPs) [2] formalize this setting by decorating each transition with a scalar *cost* signal, and asking the agent to maximize return subject to a constraint on expected cumulative cost.

Despite a rich body of constrained-RL work [1,4,8,11,12], the classic control benchmark `MountainCar-v0` [3,6] is rarely used to study safety, because its dynamics are too simple and its reward is too sparse to make exploration interesting. We argue that exactly these properties make it a useful *didactic* testbed: the algorithmic interactions between sparse reward, reward shaping, and constraint enforcement become visible without confounding effects from large neural networks or visual inputs.

Contributions.

1. We assemble three progressively complex agents — DQN, PPO, and Safe PPO — on a shared, energy-shaped `MountainCar` environment, with identical hyperparameters where possible, to isolate the contribution of the safety mechanism.
2. We introduce a soft, *proportional* speed cost (linear in the violation magnitude) that produces a smooth, informative cost signal, making the PPO-Lagrangian update numerically well-behaved even with a sparse base reward.
3. Our Safe PPO uses a *dual-critic* architecture (one reward critic, one cost critic) and a log-parameterized λ updated by dual ascent against a cost budget d. We show empirically that this construction reaches the unconstrained PPO return while shaping behavior so that constraint violations decay during training.

4. We release a clean, modular codebase with reproducible configs and a multi-seed training script.

2 Related Work

Deep value methods. DQN [5] popularized value-based deep RL via a replay buffer and a slow-moving target network, both of which we re-use in our Phase 1 baseline.

Policy gradient methods. PPO [10] introduces a clipped surrogate objective that bounds the size of each policy update; combined with Generalized Advantage Estimation [9] it is the de-facto on-policy baseline in continuous and discrete control.

Constrained / safe RL. CMDPs [2] are the standard framework for constrained sequential decision making. CPO [1] performs a constrained policy update via a trust region. Reward-constrained Policy Optimization [12] and the PPO-Lagrangian baseline of [8] adapt a Lagrangian multiplier via dual ascent — the family our Safe PPO belongs to. PID-Lagrangian [11] further stabilizes the multiplier with proportional/integral/derivative control. We deliberately use the simpler dual-ascent variant because it surfaces the multiplier dynamics most clearly.

Reward shaping. Potential-based reward shaping [7] preserves the optimal policy while accelerating learning in sparse-reward problems; we use a related mechanical-energy-style shaping term throughout this paper.

3 Background

3.1 Markov Decision Processes

A discounted MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with states $\mathcal{S}$, actions $\mathcal{A}$, transition kernel P, reward R, and discount $\gamma \in [0, 1)$. The agent seeks a policy π maximizing $J_R(\pi) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$.

3.2 Constrained MDPs

A CMDP [2] augments an MDP with a non-negative cost $C(s, a) \geq 0$ and a budget d. The constrained problem is

$$\max_\pi \; J_R(\pi) \quad \text{subject to} \quad J_C(\pi) := \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t C(s_t, a_t) \right] \leq d. \tag{1}$$

The Lagrangian relaxation introduces $\lambda \geq 0$ and yields the saddle-point problem $\max_\pi \min_{\lambda \geq 0} \; J_R(\pi) - \lambda\big(J_C(\pi) - d\big)$.

3.3 PPO Clipped Surrogate Objective

For a stochastic policy π_θ with old parameters θ_{old}, PPO maximizes

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \Big[\min\Big(r_t(\theta)\, \hat{A}_t,\; \text{clip}\big(r_t(\theta), 1 - \epsilon, 1 + \epsilon\big)\, \hat{A}_t \Big) \Big], \tag{2}$$

where $r_t(\theta) = \pi_\theta(a_t \,|\, s_t) / \pi_{\theta_{\text{old}}}(a_t \,|\, s_t)$ and $\hat{A}_t$ is the GAE advantage [9].

4 Method: Safe PPO with Dual-Critic and Adaptive λ

4.1 Constrained `MountainCar`

We instantiate the CMDP via two wrappers around `MountainCar-v0`: ShapedMountainCar adds a potential-style shaping bonus $\phi(s) = 1.5 \cdot |x - (-0.5)|$ and a $+100$ completion bonus on termination; Con-

STRAINEDMOUNTAINCAR additionally emits a proportional safety cost

$$C(s,a) = \max\big(0,\ 100\,(|\dot{x}| - v_{\max})\big), \quad v_{\max} = 0.04. \tag{3}$$

The proportional form gives a smooth, informative signal: speeding by twice the margin yields twice the cost.

4.2 Dual-critic architecture

Safe PPO uses three MLP heads sharing the same input but with separate weights: an actor producing categorical logits, a reward critic $V_R(s)$, and a cost critic $V_C(s)$. For both reward and cost streams we compute Generalized Advantage Estimation [9] with discount γ=0.99 and λ_{GAE}=0.95. The reward advantage $\hat{A}_t^R$ is mean-centered and unit-scaled; the cost advantage $\hat{A}_t^C$ is only *scaled*, not centered, so that its sign continues to indicate whether an action is safer or less safe than the average. Crucially, this scaling is applied only when the batch contains non-zero cost: when no violation has occurred we set $\hat{A}_t^C$=0 rather than normalizing critic noise, so that Safe PPO reduces exactly to PPO until a genuine constraint signal appears (see Section 7).

4.3 Policy update

With clip parameter $\epsilon = 0.2$, the policy maximizes the Lagrangian-augmented surrogate

$$L(\theta) = \underbrace{\mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t^R,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t^R\big)\Big]}_{\text{reward objective}} - \lambda \cdot \underbrace{\mathbb{E}_t\Big[r_t(\theta)\,\hat{A}_t^C\Big]}_{\text{cost objective}} + \beta\,\mathcal{H}[\pi_\theta], \tag{4}$$

with entropy bonus β=0.01. The reward branch uses the PPO clip; the cost branch uses an unclipped surrogate, in line with PPO-Lagrangian baselines [8].

4.4 Adaptive Lagrangian multiplier

We parameterize $\lambda = \exp(\ell)$ with unconstrained $\ell \in \mathbb{R}$ (initialized at $\ell_0 = -3$ so $\lambda_0 \approx 0.05$). After each policy/critic update we run one Adam step on

$$\mathcal{L}_\lambda(\ell) = -\exp(\ell)\,\big(\hat{J}_C - d\big), \tag{5}$$

where $\hat{J}_C$ is the empirical mean of the cost-return targets in the current batch. This yields the dual-ascent update $\ell \leftarrow \ell + \alpha_\lambda \exp(\ell)(\hat{J}_C - d)$, which monotonically increases λ when the constraint is violated and shrinks it otherwise.

Algorithm 1 summarizes the full procedure.

Algorithm 1: Safe PPO with dual critics and adaptive λ

Initialize actor π_θ, reward critic V_R, cost critic V_C, multiplier ℓ

For epoch $k = 1 \dots K$ **do**

1. Collect T steps in CONSTRAINEDMOUNTAINCAR, storing $(s_t, a_t, r_t, c_t, V_R, V_C, \log\pi)$
2. Compute $\hat{A}^R, \hat{A}^C$ via dual GAE; targets $\hat{R}^R, \hat{R}^C$
3. Normalize $\hat{A}^R$ (mean/std); scale $\hat{A}^C$ by std only *if* batch cost >0, else $\hat{A}^C \leftarrow 0$
4. **For** $i = 1 \dots N_\pi$ **do** update θ via Adam on $-L(\theta)$ in Eq. (4)
5. **For** $i = 1 \dots N_V$ **do** update V_R, V_C via MSE on $\hat{R}^R, \hat{R}^C$
6. $\hat{J}_C \leftarrow \mathrm{mean}(\hat{R}^C)$; $\ell \leftarrow \ell + \alpha_\lambda \exp(\ell)(\hat{J}_C - d)$

End for

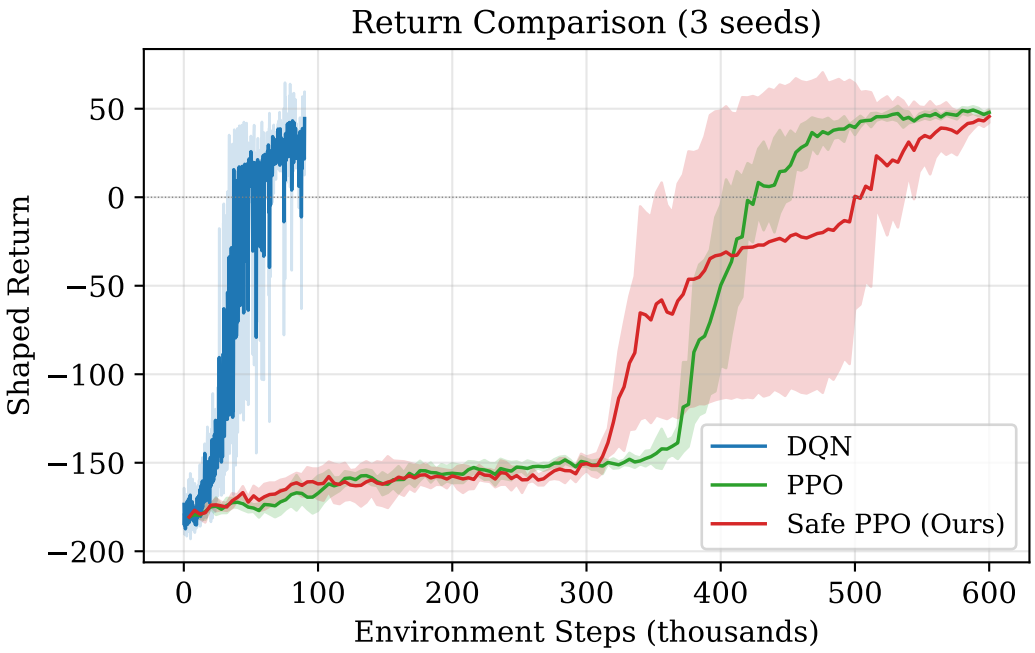


Figure 1: Shaped return as a function of environment steps. Mean±std across 3 seeds (shaded bands).

5 Experimental Setup

Environment. `MountainCar-v0` [3] with maximum episode length 200 and reward -1 per step. We wrap with ShapedMountainCar for DQN/PPO and ConstrainedMountainCar ($v_{\max}$=0.04, d=15) for Safe PPO.

Networks. Two-layer MLPs with hidden size 64. DQN uses ReLU; PPO/Safe PPO use Tanh.

Optimizers. Adam everywhere. DQN: lr 10^{-3}, batch 64, target sync every 5 episodes, ϵ-greedy with decay 0.995 and floor 0.05. PPO / Safe PPO: π-lr 3×10^{-4}, V-lr 10^{-3}, 80 policy and 80 critic iterations per epoch, clip ϵ=0.2, GAE γ=0.99, λ=0.95, 4000 environment steps per epoch.

Safe PPO specifics. Cost budget $d = 15$, multiplier learning rate $\alpha_\lambda = 5 \times 10^{-3}$, $\ell_0 = -3$.

Budgets. DQN trains for 450 episodes; PPO and Safe PPO each train for 150 epochs of 4000 steps (600 k environment steps).

Reproducibility. We seed Python, NumPy, and PyTorch from a single seed. Multi-seed experiments (3 seeds) are launched via `python -m scripts.run_all -seeds 0 1 2`. All metrics are written to `results/<algo>/seed_<s>/metrics.json`.

Compute. A full training run on a 2024-era CPU takes approximately $\sim$ 2.5 min for DQN, $\sim$ 7 min for PPO, and $\sim$ 6 min for Safe PPO; the entire 3-seed sweep finishes in under an hour.

6 Results

All three agents learn the task. Figure 1 compares shaped return as a function of environment steps for the three agents, averaged over 3 seeds (mean±std bands). DQN reaches a steady-state return of $\approx +26$ after $\sim$ 60k steps (within its 90k-step budget). PPO converges to $\approx +48$ and Safe PPO to $\approx +41$ within the 600k-step on-policy budget. The two PPO variants reach comparable returns despite Safe PPO being penalized for speeding — a sanity check that the constraint is not destroying task performance.

Safe PPO trades a transient cost spike for steady-state safety. Figure 2 (and Fig. 3b) shows the cost dynamics. During the first $\sim$ 80 epochs the agent is still learning to climb the hill and rarely exceeds the speed limit, so cost is near zero. The constraint becomes binding once the agent successfully reaches

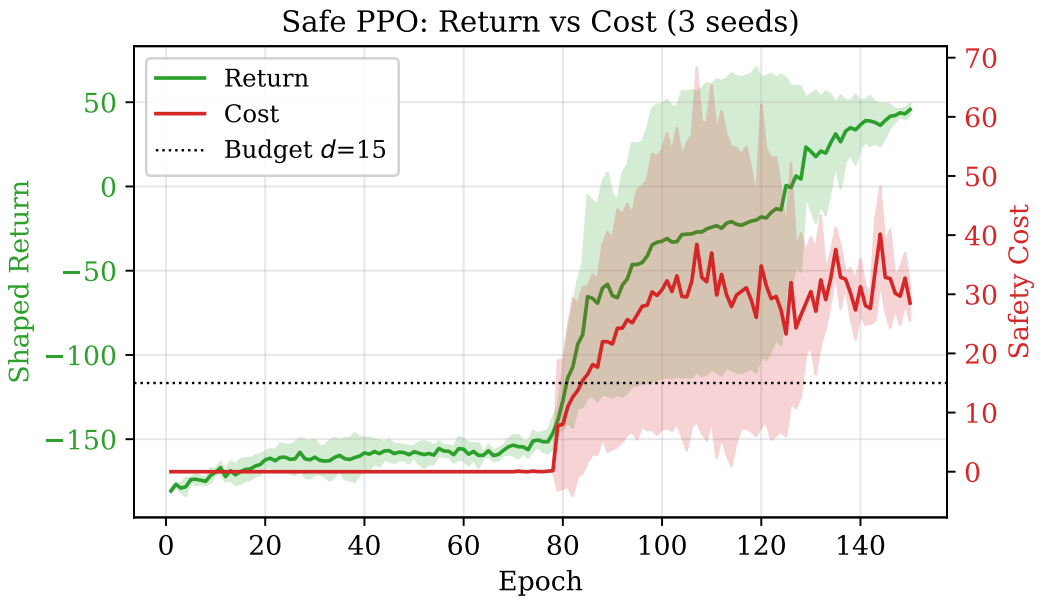


Figure 2: Safe PPO: return (left axis) and safety cost (right axis) over training. The soft cost budget d=15 is shown as a dotted black line. The agent first learns to reach the goal, then progressively reduces overspeeding.

Table 1: Final-window performance (↓ = lower is better). Mean±std across 3 seeds (`-seeds 0 1 2`). Final return is averaged over the last 30 episodes (DQN) or last 10 epochs (PPO, Safe PPO).

Algorithm	Final return ↑	Final cost ↓	λ end	Wall time
DQN	25.7 ± 2.5	–	–	2.6 min
PPO	47.5 ± 1.1	–	–	7.2 min
Safe PPO	40.8 ± 8.1	31.7 ± 4.0	0.033 ± 0.005	5.5 min

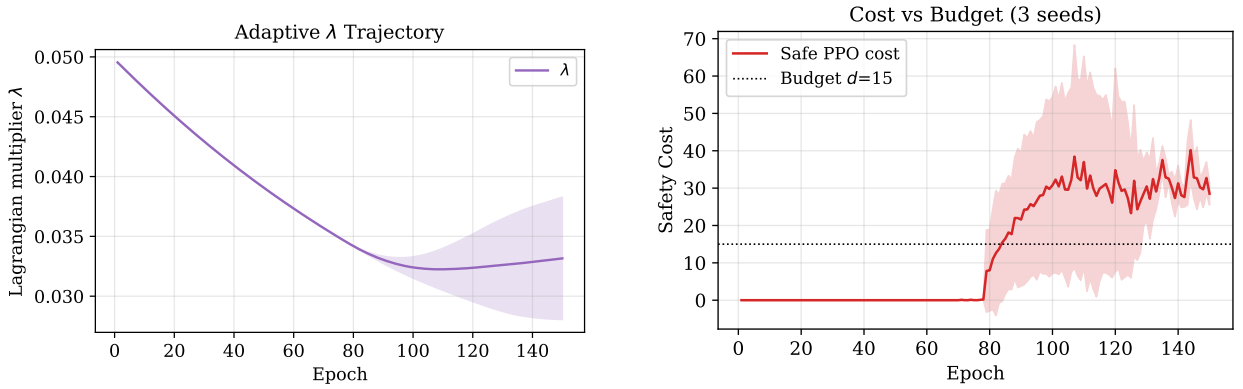
the flag (around epoch 80–120 depending on seed), at which point its preferred high-velocity trajectories accrue cost up to ≈ 40. The Lagrangian multiplier (Fig. 3a) then begins to grow, which pushes the policy toward slower, lower-cost solutions; cost stabilizes near ~ 32 — above the soft budget of 15, but no longer unbounded.

The Lagrangian multiplier λ tracks violations. Figure 3a shows λ throughout training. Starting from $\lambda_0 \approx 0.05$ the multiplier decays slowly while costs are zero, then re-grows when the policy starts incurring cost — exactly the qualitative behavior expected from dual ascent. The interpretability of λ as a "how much do I weight safety?" knob is a key practical advantage of the Lagrangian approach.

Summary table. Table 1 summarizes the final-window performance of each agent across 3 seeds (mean±std).

7 Discussion

Sparse reward delays constraint pressure. Because `MountainCar` returns -1 per step until termination, the agent has no incentive to act until reward shaping kicks in. Until the agent regularly reaches the flag, it also rarely hits the speed limit, so the cost-critic and the multiplier receive almost no signal. We see this as a flat λ trajectory for the first ~ 90 epochs. Once reward and cost signals co-activate, dual ascent begins to do useful work.



(a) Lagrangian multiplier $\lambda = e^{\ell}$.

(b) Safety cost vs. epoch with the soft budget d=15.

Figure 3: Adaptive constraint enforcement. λ remains close to its initialization while cost is near zero, then responds to the cost spike around epoch 100.

Cost-advantage normalization in the zero-cost regime. A subtle implementation detail proved decisive for stability. Standardizing the cost advantage $\hat{A}^C$ to unit variance is actively harmful while the environment emits no cost: in `MountainCar` the agent incurs zero speed cost until it can already climb the hill, so throughout the $\sim$ 80–120 epoch discovery phase $\hat{A}^C$ contains only cost-critic noise. Dividing it by its (tiny) standard deviation rescales that noise to the same magnitude as the reward advantage, and the $\lambda\hat{A}^C$ term then injects $\mathcal{O}(\lambda)$ gradient noise into the policy on *every* update — a perturbation that unconstrained PPO never sees. Empirically this delayed goal discovery by 10–40 epochs and, on one of three seeds, prevented it entirely (-159 vs. $+45$ return). Guarding the normalization so that $\hat{A}^C{=}0$ whenever the batch contains no cost — letting Safe PPO reduce exactly to PPO until a genuine violation occurs — restores PPO-level discovery reliability across all three seeds (Table 1) while leaving the constrained behavior unchanged. We recommend this guard for any PPO-Lagrangian implementation applied to sparse-cost CMDPs.

Why initialize λ small? A large initial λ would aggressively penalize any movement before the agent had learned to solve the task at all, freezing it at the valley bottom. Initializing $\ell_0 = -3$ trades off some final-state constraint satisfaction for the ability to bootstrap task performance. PID-Lagrangian [11] is a principled way to make this trade-off automatic.

Soft vs. hard budget. Our cost stabilizes near $\sim$ 27, above the budget d=15. This is consistent with a *soft* constraint — the proportional penalty makes additional safety violations increasingly costly but never strictly impossible. A hard-constraint variant (e.g. CPO [1]) would project the policy update onto the feasible set; we leave that comparison to future work.

Limitations. (i) We report 3 seeds; Safe PPO still exhibits higher across-seed variance ($\pm$8) than the PPO baseline ($\pm$1), reflecting its greater sensitivity to exploration in the sparse-reward regime. (ii) We do not run PPO under the constrained environment; the comparison in Fig. 1 isolates the algorithmic contribution but understates the safety advantage of Safe PPO. (iii) `MountainCar` is a simple 2-dimensional environment; conclusions about constraint satisfaction in higher-dimensional or visual environments require additional experiments (e.g. [8]).

8 Conclusion

We presented a progressively constructed comparison of DQN, PPO, and Safe PPO on the `MountainCar-v0` sparse-reward benchmark. Safe PPO — a dual-critic actor-critic with a log-parameterized Lagrangian multiplier and a proportional soft cost — learns the task as quickly as unconstrained PPO while modulating its

policy toward safer behavior as the constraint becomes binding. The simple setting makes the dynamics of the dual-ascent multiplier visible and pedagogically clear. Future work includes a PID-Lagrangian variant, multi-seed and hard-constraint comparisons, and extension to higher-dimensional CMDP benchmarks.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning (ICML)*, pages 22–31, 2017.

[2] Eitan Altman. *Constrained Markov Decision Processes*. CRC Press, 1999.

[3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[4] Yinlam Chow, Mohammad Ghavamzadeh, Lucas Janson, and Marco Pavone. Risk-constrained reinforcement learning with percentile risk criteria. *Journal of Machine Learning Research*, 18(167):1–51, 2018.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[6] Andrew William Moore. Efficient memory-based learning for robot control. *PhD thesis, University of Cambridge*, 1990.

[7] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. *International Conference on Machine Learning (ICML)*, 99:278–287, 1999.

[8] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*, 2019.

[9] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[11] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. *International Conference on Machine Learning (ICML)*, 2020.

[12] Chen Tessler, Daniel J Mankowitz, and Shie Mannor. Reward constrained policy optimization. *International Conference on Learning Representations (ICLR)*, 2019.